

# Tema 0: Introducción a R y Positron (Posit)

Técnicas para 'Big Data' en Economía - Curso 2026/27  
Universidad de Alicante

Pedro Albarrán

Dpto. de Fundamentos del Análisis Económico. Universidad de Alicante



# Contenidos I

1 Conceptos básicos

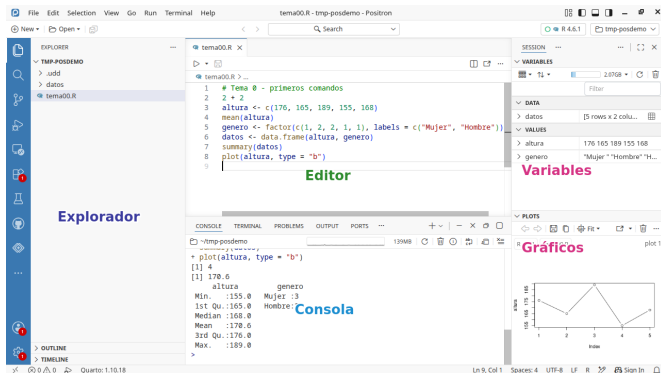
2 Objetos en R

3 Extendiendo R

# Introducción

- **R** es el lenguaje: ejecuta instrucciones directamente en una consola (es *interpretado*)
- **Positron** es el entorno de trabajo (IDE): derivado de VS Code, uno de los editores más usados
  - ▶ Misma interfaz y manejo, con añadidos específicos para el análisis de datos en R y Python
  - ▶ Se puede usar VS Code “puro” de forma muy similar
- Instalación: página Puesta a punto de la web (si no lo tenéis, LO ANTES POSIBLE)
- Este tema: de los primeros comandos hasta empezar a trabajar con datos; el material de cada sesión, siempre en Contenidos

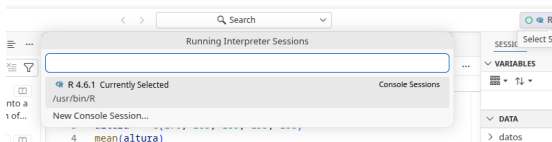
# Positron



- **Editor** (scripts) y **Consola** (ejecutar comandos y ver resultados); a la derecha, **Variables** (objetos en memoria) y **Gráficos**; a la izquierda, el **Explorador** de archivos

# Empezando: elegir el intérprete de R

- Al abrir Positron aparece la zona de **consola** (panel inferior), PERO hay que **elegir el intérprete**: pulsad en el selector de la esquina superior derecha y elegid **R** (versión 4.6.x)



- OJO: si aparece **Python**, NO es un error vuestro, pero NO es lo que usamos: cambiad a R en ese mismo selector (*New Console Session...*)

# La Consola

- Escribimos *comandos* en la **consola** y se ejecutan pulsando Enter:

- ▶ La tecla de tabulador  ofrece opciones de autocompletado
- ▶ Ejecutar algo que no es un **comando de R** devuelve un error

```
2 + 2
3 * (1 - 4)^2
sqrt(log(5/2))
pi
hola
```

# Archivos de guion (“scripts”)

- Es preferible incluir varios comandos en un archivo de texto para ejecutarlos
- Se puede replicar el proceso de cálculos paso a paso (no como Excel)
- Creamos un nuevo archivo en el menú *File > New File...* eligiendo *R File*
- Guardamos el archivo en *File > Save* (atajo **Ctrl + S**), eligiendo un directorio y nombre con extensión “.R” (por defecto) o “.r”
- Desde el editor, **Ctrl+Enter** envía a la consola la línea actual o las líneas seleccionadas (en MacOS, la tecla **Command** en lugar de **Ctrl**)
- En un archivo de guion (guardado), Positron marca las líneas con error y muestra el mensaje de error al pasar el puntero



The screenshot shows the Positron R editor with a file named 'miscript.R'. The code in the editor is as follows:

```
1 #  
2 Syntax error  
3 View Pro... No quick  
4 2 * 2  
5 sqrt(log(5/2))  
6 p  
7
```

A tooltip is visible over the 'Syntax error' message on line 2, displaying the text 'View Pro... No quick'. The cursor is positioned at the end of line 6.

# Trabajar con ficheros de guion

- Cada comando es “una” línea y se ejecutan secuencialmente
- Un comando se puede extender visualmente más de una línea hasta completarse: p.e., hasta cerrar los paréntesis.
  - ▶ Escribimos `log(`, en otra línea y ejecutamos: la consola cambia de `>` a `+`
  - ▶ No hace “nada” esperando que completemos el comando.
- El carácter `#` marca el inicio de un **comentario**: lo que sigue se “ignora” (no se ejecuta) en R

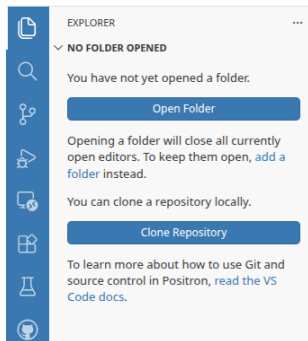
```
# Pueden ir al principio de la línea  
2 + 2 # o después de una instrucción
```

- Comentar es un buen hábito: ayuda a entender/recordar qué hacemos
- Notad que Positron tiene *resaltado de sintaxis*: distinto color para comentarios, números, funciones, etc.



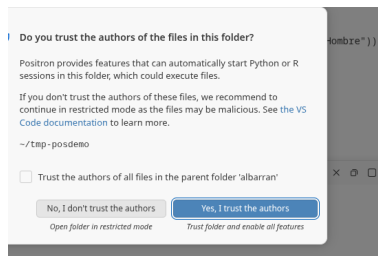
# Directorio de Trabajo. Proyectos.

- Conviene **organizar los archivos** relacionados con un mismo tema en una estructura de (sub)directorios a partir de un **directorio de trabajo** principal
- En la programación moderna (Positron, VS Code y la mayoría de editores actuales), un **proyecto** ES simplemente una **carpeta**
- Cread una carpeta para el curso (p.e. *AnalisisDatos*) con subcarpetas para datos, código, etc., y abridla con *File > Open Folder*
- Sin carpeta abierta, el **Explorador** (barra izquierda) solo ofrece precisamente eso: *Open Folder*



# La primera vez que abrí la carpeta

- Al abrir una carpeta por primera vez, Positron pregunta si **confiáis** en sus archivos: pulsad **“Yes, I trust the authors”** (es vuestra carpeta: la acabáis de crear)
- Si contestáis que no, Positron queda en **modo restringido**: sin consola de R, sin resaltado, sin casi nada; se arregla pulsando en *Restricted Mode* (abajo a la izquierda) y confiando la carpeta
- Con la carpeta abierta, esa carpeta es vuestro **directorio de trabajo**: R busca y guarda los archivos ahí (usad rutas RELATIVAS, como veremos)
- Al abrir Positron se recupera la última carpeta usada; con *File > Open Recent* podéis cambiar entre carpetas (p.e., otra asignatura u otro proyecto)



## Proyectos (cont.)

- Para trabajar con un archivo, usamos la **ruta relativa** al directorio de trabajo:
  - ▶ si están en el raíz del directorio: `codigo.R`, `misdatos.Rdata`
  - ▶ si están en un subdirectorio, indicamos la ruta separando directorios por `/`:  
`datos/ventas.Rdata`, `datos/ano2020/ingresos.Rdata`
- Las rutas relativas son otra idea general de la programación: el proyecto funciona igual en cualquier ordenador (la ruta completa `C:/Users/minombre/...` solo existe en el vuestro)

# El Explorador y la barra lateral

- El **Explorador** (icono de archivos en la barra lateral izquierda) ofrece una forma visual de abrir, crear, copiar, mover o eliminar archivos y carpetas
- El Explorador solo muestra **lo que está DENTRO de la carpeta abierta**: lo de fuera no se ve ni se puede tocar desde aquí
  - ▶ Por eso hay que pensar QUÉ pertenece al proyecto (datos, scripts...) y traerlo a la carpeta (arrastrándolo al Explorador, o copiándolo con el explorador del sistema)
- Esa barra lateral tiene mucho más (extensiones, asistente de IA, control de versiones, búsqueda): NO lo necesitáis por ahora; la ayuda y la IA integradas las iremos usando poco a poco
- Evitad caracteres “raros” (acentos, espacios, etc.) en directorios y ficheros
- NOTA. El Explorador de Archivos de Windows y Finder de MacOS no muestran por defecto las extensiones de los archivos.
  - ▶ Puede ser confuso para distinguir entre dos archivos con el mismo nombre y diferente extensión: `proyecto.R` y `proyecto.qmd`



# Funciones en R

- Las expresiones que aceptan **argumentos** se denominan funciones.

```
exp(2)
ceiling(5.2)
```

- Algunos argumentos son obligatorios, otros tienen valores por defecto que se pueden omitir
- Los argumentos se pueden especificar por nombre o por orden.

```
log(2, base=2)
log(2, 10)
log(base = 10, x = 2)
```

- ¿Cómo sabemos la manera de usar una función (ej. argumentos necesarios) o comando de R?

# Ayuda en R y Positron

- Positron tiene autocompletado (tecla tabulador) y ayuda flotante para funciones y otros elementos de R

```
type 'contributors()' for more information and  
'citation()' on how to cite R or R packages in |  
  
Type 'demo()' for some demos, 'help()' for on-l  
'help.log(x, base = exp(1))' interface to  
Type 'a numeric or complex vector.  
  
> log()
```

- Positron también tiene un panel de **Ayuda** (Help): F1 sobre una función, o ?min / help(min) en la consola

# La IA como ayuda (usada con cabeza)

- Positron incluye un **asistente de IA** (icono de chat en la barra lateral), que se conecta a proveedores como Copilot o Claude; también en VS Code, y siempre están las IAs externas (ChatGPT, Gemini...)
- Usos LEGÍTIMOS que os harán mejores: pedir que os **explique** un mensaje de error, recordar la sintaxis de una función, entender un código ajeno
- Uso que os hará PEORES: dejar que **reescriba vuestro código sin entenderlo**: no aprendéis, no detectáis sus errores... y en la evaluación se nota
- La disciplina es la misma que el *pensamiento algorítmico* del primer día:
  - ① especificar CON PRECISIÓN el problema (también al preguntar a una IA)
  - ② descomponerlo en pasos
  - ③ COMPROBAR el resultado (¿hace lo que quería? ¿con qué casos falla?)
- Ante un error: leer el mensaje, localizar la línea, formular una hipótesis, probar; la IA ayuda a *entender*, la solución tenéis que entenderla vosotros: copiar-pensar-adaptar



# El operador de asignación

- El operador `<-` almacena un contenido en un *objeto* con un nombre,<sup>1</sup> que incluye letras, números y algunos caracteres especiales ("`.`", "`_`")

```
x <- 2*3      # asignación, no muestra resultado
x             # ejecutamos mostrar la variable
print(x)
(x <- 2)      # asignación e impresión a la vez
```

- R es "case-sensitive": `x` y `X` son dos objetos distintos
- Los objetos asignados pueden usarse posteriormente, p.e., para generar otros a partir de ellos

```
y <- x + 5    # asignamos y a partir del VALOR de x
(x <- x*3)    # re-asignamos x a partir de ella misma
y            # NO cambia (en Excel, sí)
```

---

<sup>1</sup>También se puede asignar con `=`

# El Espacio de Trabajo en R

- El espacio de trabajo es el conjunto de objetos activos en memoria, resultado de todos los comandos ejecutados previamente
- En Positron, el panel **Variables** muestra los objetos y su valor
- Las funciones `ls()` y `rm()` muestran y eliminan respectivamente objetos del espacio de trabajo
- Borramos *todos* los objetos desde el panel Variables o con el comando

```
rm( list=ls() )  # eliminar todos los objetos
rm(y, x)         # eliminar solo algunos objetos
```

- Guardar el contenido del entorno de trabajo al cerrar es **innecesario**: ejecutando los comandos guardados en un archivo `.R` recuperamos los objetos

# Mensajes de “Error” y “Warning”

- En programación, cometer **errores** es **normal**
- En muchos errores, R se “quejará” mostrando **mensajes** en rojo
  - ▶ *Aviso*: R ofrece un resultado (y continuará al siguiente comando), PERO indica que puede haber algo “no deseado”
  - ▶ *Error*: para la ejecución, *sin resultado*, e “informa” de la razón
- Algunos mensajes son claros, pero otros requieren más investigación: pegarlos a una IA y pedir que os los EXPLIQUE es un buen uso; que os reescriba el código sin entenderlo, no
- Peor que un mensaje de error: escribimos (copiamos) un código que funciona pero no hace lo que queremos...
- El ordenador NO se equivoca: hace lo que le pedimos según unas reglas bien definidas por R, que *debemos conocer*
  - ▶ Sed cuidadosos, pensad y **entended** cada paso del código



## Un ejemplo para toda esta parte

- A partir de aquí nos acompaña un mini-caso: las **fichas de clientes** de un pequeño gimnasio: nombre, altura, peso, si es socio, gasto mensual, género, satisfacción
- Iremos como en un análisis real: de una variable suelta (un **vector**) a la información cualitativa (**factores**) y a la tabla completa (**data frame**)
- Habrá varias **PRÁCTICAS en clase** sobre este caso: implementar vosotros es donde se aprende
- NO hay que memorizar cada comando: hay que entender la IDEA de cada uno; los detalles se consultan (Ayuda, IA)
- Y aprenderemos a **leer los fallos**: los mensajes de error, y los casos peores en que no hay mensaje pero el resultado no es el que queríamos

# Cómo se entrega el trabajo de clase

- Lo que se entrega es el trabajo de las **PRÁCTICAS** de este tema (el resto de cosas que probéis o veamos en pantalla, no)
- Para este tema, ese trabajo va en un guion propio: Tema00\_DNI.R (con VUESTRO DNI: p.e., Tema00\_12345678X.R), en la carpeta del curso
  - ▶ cada tema tendrá su propia entrega (y su formato); en algunos temas podrá haber más de una
- Si una tarea pide *contestar* algo (no solo código), escribid la respuesta como TEXTO en un comentario del guion
- Cuando se pida, se sube mediante un formulario cuyo enlace se dará en clase en ese momento
- Cuenta el TRABAJO, no la perfección: se valora que lo habéis ido haciendo y razonando



# Tipos de Objetos en R

- TODO en R es un objeto, cada uno con **distintas propiedades** y, por tanto, distintas formas de trabajar con él
- Además de las funciones, los principales objetos con los que trabajaremos son:
  - 1 vectores
  - 2 factores
  - 3 conjuntos de datos (“data frames”)
  - 4 listas
- Estos objetos pueden contener varios tipos de datos o variables:
  - ▶ entero
  - ▶ numérico (números reales)
  - ▶ lógico (valores verdadero/falso)
  - ▶ caracteres

# Vectores

- Un vector es una secuencia de datos elementales, creados con el operador “c()” (combinar)
- Empezamos nuestras fichas: cada *variable* de los 5 clientes es un vector

```
altura <- c(176, 165, 189, 155, 168)           # numérico
cliente <- c("Jose", "Maria", "Juan", "Elena", "Rosa") # caracteres
socio <- c(TRUE, FALSE, TRUE, TRUE, FALSE)      # lógico
```

- Podemos crear vectores a partir de otros vectores o usando comandos

```
z <- c(altura, 170)   # añadir un dato
x <- rep(1, times=4)
y <- seq(from = 10, to = 1, by = -1)
z <- 1:10              # equivale a z <- seq(1,10,1)
```

- Un vector sólo puede contener objetos de un **único tipo** elemental, que podemos conocer con `str()` o `class()` (el panel Variables muestra los valores)



## Vectores (cont.)

- Si se mezclan tipos distintos, R busca una clase que “acomode” a todos

```
vcr <- c("lunes", 2)
```

- Forzamos que un objeto sea tratado con una clase concreta, con `as.integer()`, `as.numeric()`, `as.character()` y `as.logical()`
  - ▶ Si no se puede convertir a número, devuelve NA (con un “warning”)
- NO se pueden realizar operaciones incompatibles entre clases
  - ▶ Cuidado con las comillas: NO es lo mismo un objeto (su contenido) que el carácter de su nombre

```
a <- 4  
c <- 'a' + 1
```

- Los vectores pueden tener *nombres* (una “etiqueta” única para cada elemento): un vector de caracteres de la misma longitud asignado con `names()`

```
names(altura) <- cliente # etiquetamos cada altura con su cliente  
altura
```



# Aritmética de vectores

- La mayoría de los operadores se aplican *elemento-a-elemento*: p.e., el IMC de CADA cliente a partir de su peso y su altura

```
peso <- c(75, 85, 70, 60, 80)
imc  <- peso / (altura/100)^2 # un cálculo, 5 resultados
```

- Con diferentes longitudes, se repite el vector corto cuanto sea necesario

```
altura + c(1, 2, 3, 4, 5)
altura + 1 # se "recicla" el 1
```

- Algunas **funciones** relevantes

```
length(altura) # longitud
sort(altura)   # ordenar
max(altura)    # máximo
min(altura)    # mínimo
sum(peso)      # suma
```

```
mean(altura)   # media
var(altura)    # varianza
table(socio)   # frecuencias
summary(altura) # estadísticos
```

# Vectores lógicos

- Obtenemos un objeto lógico enunciando una relación que puede ser cierta o falsa , como comparaciones básicas de igualdad o desigualdad: p.e., ¿qué clientes miden más de 165 cm?

```
1 == 1 # TRUE
1 != 3 # TRUE
1 > 2  # FALSE
```

```
a <- 3
a >= 3      # TRUE
a + 1 <= 10 # FALSE
```

- Combinamos varios enunciados con operadores Y (&), O (|) y NO (!)
- Para conjuntos, `x %in% Y` es cierto cuando `x` es un elemento de `Y`

```
altura > 165 # ¿qué clientes miden más de 165 cm?
altura >= c(175, 165, 195, 165, 168) # elemento a elemento
altura > 160 & altura <= 180
altura < 160 | altura >= 180
c(165,179) %in% altura
condicion <- !(altura <= 170)
```



# Acceso a los elementos de un vector

- Se utiliza el operador `[]` (paréntesis cuadrado)<sup>2</sup> y

❶ **Posiciones** de los elementos, usando un vector de enteros

```
altura[3]  
altura[c(1,3,5)]
```

- Con enteros negativos, indicamos posiciones que NO queremos

```
altura[-c(2,4)]
```

❷ **Condición** que satisfacen los elementos, usando un vector lógico

```
altura[altura > 180 | altura < 160]
```

❸ (Si lo tienen) **Nombres** de los elementos, usando un vector de caracteres

---

<sup>2</sup>También con `[[ ]]`

# PRÁCTICA en clase: las fichas (1)

Con los vectores `cliente`, `altura`, `peso` y `socio` de nuestras fichas:

- 1 Cread el vector `gasto` con el gasto mensual de cada cliente: 45, 60, 30, 75, 50
- 2 Calculad el gasto medio mensual y el gasto TOTAL anual del negocio
- 3 Obtened el gasto de los clientes que NO son socios; ¿su media es mayor o menor que la de los socios?
- 4 ¿QUÉ clientes (sus nombres) gastan más de 40 al mes?
- 5 Elena se hace socia: corregid el dato
- 6 Llega un cliente nuevo (Luis, 172 cm, 68 kg, no socio, gasto 55): actualizad los vectores. ¿Qué puede salir mal si olvidáis uno?
- Primero SIN ayuda; luego comparad con lo que sugiere una IA: ¿entendéis las diferencias?



# Factores

- En nuestras fichas, el género o la satisfacción son **información cualitativa**: se suele codificar como texto o números, pero NO tiene sentido numérico (ni de “texto”): *representan* clases o **categorías**
- Para destacar la naturaleza distinta de estos datos, existe un tipo de objeto específico en R: los **factores**
- Además de otras ventajas que veremos, permiten separar la representación original de las categorías (niveles) de cómo queremos mostrarlas (etiquetas)

```
genero    <- c(2, 1, 2, 2, 2)
genero_f  <- factor(genero, levels = c(1, 2),
                    labels = c("Mujer", "Hombre"))
```

- Se asocia nivel 1 con “Mujer”, 2 con “Hombre”, etc.
- Las operaciones con factores se realiza con las etiquetas, no con los niveles

```
genero_f == 1      # NO existe valor 1
genero_f == "Mujer"
```



## Factores (cont.)

- También podemos usar `as.factor()` para convertir un vector en un factor
- PERO es conveniente especificar niveles y etiquetas: si no, R usa como niveles los valores únicos **ordenados** (numérica o alfabéticamente) y como etiquetas esos mismos valores

```
g <- factor(genero) # as.factor(genero) hace lo mismo
g                  # niveles: "1" < "2"
```

- Las etiquetas resultantes ("1", "2") no son informativas: no sabemos cuál es "Mujer" y cuál "Hombre"
- También podemos tener factores con orden con la opción `order = TRUE` y enumerando los niveles en orden

```
satisf <- c("A", "B", "A", "B", "M")
satisf_f <- factor(satisf, order = TRUE,
                  levels = c("B", "M", "A"),
                  labels = c("Bajo", "Medio", "Alto"))
```



## El tipo importa: `summary()` con vector y factor

- La función `summary()` devuelve los principales estadísticos descriptivos de un vector numérico

```
summary(altura)
```

- Para información cualitativa, la media y otros estadísticos no tienen sentido

```
summary(genero)
genero <- c(1, 20, 20, 1, 1) # dos categorías igualmente
summary(genero)
```

- La función `summary()` ofrece resultados diferentes según el tipo de objeto (porque tiene distintas propiedades)

```
summary(genero_f)
```

## PRÁCTICA en clase: las fichas (2): el tipo importa

- 1 Cread `satisf <- c(2, 3, 2, 1, 3)` (la satisfacción de los 5 clientes: 1=Baja, 2=Media, 3=Alta) y `summary(satisf)`: ¿tiene sentido “una satisfacción media de 2.2”?
- 2 Convertirlo en factor **ordenado** con esas etiquetas y `summary()`: ¿qué cambia y por qué?
- 3 Probad `satisf_f >= "Media"` para localizar clientes satisfechos; ¿funcionaría con un factor SIN orden, como el género?
- Moraleja: el MISMO dato con tipo distinto se analiza distinto; elegir bien el tipo es parte del análisis

# “Data Frames”

- Es un tipo de objetos específico para facilitar el análisis de datos: una colección de variables por columnas y observaciones por filas.
- Cada columna es un vector con un **nombre** y tipos de datos (quizás) diferentes
- Nuestras fichas de clientes son EXACTAMENTE eso: las juntamos en una tabla

```
datos <- data.frame(cliente, altura, peso, genero_f, gasto)
```

- Se visualizan con `View(datos)` (abre el **Data Explorer** de Positron, con filtros y ordenación), desde el panel Variables, o una parte con `head()`
- Seleccionamos columnas por nombre **con \$** o por nombre o posición **con [[ ]]**

```
vectAltura <- datos$altura      # objeto resultante = vector  
datos[[3]] == datos[["peso"]]
```



## “Data Frames” (cont.)

- También podemos usar `[]` para seleccionar filas y columnas por posición, nombre y/o condición lógica

```
datos1 <- datos[datos$genero_f == "Hombre", 2:3] # altura y peso de
```

- Suele ser mejor usar `subset()` que devuelve siempre un “data frame”

```
D1 <- subset(datos, altura > 165)
D2 <- subset(datos, subset = altura %in% c(176,189),
             select = c(altura, peso))
```

- Generamos nuevas variables con el vector de asignación

```
datos$altura_m <- datos$altura / 100
```

## PRÁCTICA en clase: las fichas (y 3)

- 1 Reconstruid el *data frame* `datos` con `cliente`, `altura`, `peso`, `genero_f` y `gasto`, y añadid la columna `imc`
- 2 Extraed las fichas de los clientes con `imc > 25`: ¿data frame o vector?
- 3 ¿Gastan más los hombres o las mujeres? (media de `gasto` por grupo)
- 4 Abrid la tabla con `View(datos)` (Data Explorer) y comprobad los tipos con `str()`: ¿coincide cada columna con lo que esperabais?

# Listas

- Una lista es una colección de objetos de distinto tipo (a diferencia de un vector)
- Los elementos de una lista suelen tener nombres

```
miLista <- list(saludo="hola", vec=x, lista=list(1:4, "X"),
               datos=datos)
```

- Con `[[ ]]` (por posición o por nombre) o con `$` (solo por nombre) extraemos los elementos en su clase original

```
miLista$vec
miLista[[2]] + 3
```

- También podemos usar `[]`, pero devuelve una lista
- `unlist()` convierte una lista en vector, usando la clase que pueda ajustarse a todos los objetos (elementales)



# Las listas guardan resultados de funciones

- Muchas funciones de R devuelven sus **resultados como una lista**: guardan cosas diversas (estadísticos, opciones, datos...) en un único objeto
- Por eso importa saber **ACCEDER** a sus elementos: cuando ejecutemos un contraste o una regresión, extraeremos de ahí lo que necesitemos

```
res <- t.test(altura)    # el resultado es una lista
names(res)              # qué contiene
res$p.value             # accedemos a un elemento
```

# Bibliotecas (“libraries”)

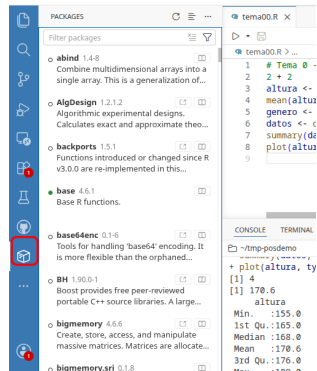
- Una biblioteca contiene nuevos objetos de R: funciones, datos, etc.
- Para instalar una nueva biblioteca (se hace **una vez**), con el comando

```
install.packages("AER")
```

- O visualmente en el panel **Packages** (icono en la barra de la izquierda): botón *Install Package*; ahí también se actualizan (filtro *Outdated*)
- La biblioteca solo está disponible si se carga en la *sesión actual*

```
library(AER)
```

- Nota: en adelante, la bibliotecas que carguemos se suponen instaladas
- El panel Packages muestra las instaladas y marca las cargadas (“attached”)



# Bibliotecas (cont.)

- El nombre completo de un objeto es `biblioteca::nombre`
  - ▶ La nombre de la biblioteca solo es necesaria si no se ha cargado o dos objetos diferentes tienen el mismo nombre en bibliotecas distintas

```
base::log(1)
log(1)
```

```
library(Hmisc)
find("units")
```

- Para mostrar todos los datos de las bibliotecas cargadas

```
data()
```

- Los podemos cargar (aparecen en el panel Variables) y obtener información detallada en la ayuda (ej., nombre de variables)

```
data("Affairs")
help("Affairs")
```

# Datos “nativos” en R

- Guardamos objetos del espacio de trabajo con `save()` (en una ruta relativa al directorio de trabajo)

```
x <- 1:20
y <- 2 * x ^ 2 + 1
save(x, file="x.RData")          # un objeto, o varios
save(x, y, file="data/xy.RData") # separados por comas
```

- Para cargar datos al espacio de trabajo, con `load()`

```
load("data/xy.RData")
```

- Nota: este tipo de archivo puede contener varios objetos, incluidas varios conjuntos de datos

# Datos en otros formatos externos a R

- Varias bibliotecas permiten trabajar con distintos formatos de datos, p.e.:
  - ① Texto, con delimitadores o de ancho fijo: `utils` (R base), `readr`
  - ② Hojas de cálculo: `readxl`, `openxlsx`
  - ③ Formatos de software estadístico: `haven`, `foreign`
- Descargad estos ejemplos (UA cloud): `renta.txt`, `sex_data.csv`, `beauty.xls`, `nsw.dta`
- Positron NO tiene menú visual de importación: los datos se cargan con **comandos** (mejor: quedan en el script y el proceso es replicable)
- PERO el **Data Explorer** de Positron permite *ver* un archivo de datos (p.e., un `.csv`) como tabla, con filtros y ordenación: doble clic en el Explorador
- `rio` es un meta-paquete (instala otras bibliotecas) para importar y exportar varios formatos de datos de forma sencilla
  - ▶ A partir de la extensión del archivo, detecta el formato y, por tanto, la biblioteca necesaria



# Importar y exportar con rio

- rio permite trabajar con el **mismo comando** para todos los formatos, pero las opciones por defecto pueden no ser adecuadas
  - ▶ En la Ayuda se incluye una presentación completa del paquete
- El comando `import()` se usa para leer los datos

```
library(rio)
sex    <- import("data/sex_data.csv")
renta  <- import("data/renta.txt")
beauty <- import("data/beauty.xls")
nsw    <- import("data/nsw.dta")      # formato Stata
```

- Podemos exportar datos a un tipo de formato con `export()`

```
export(nsw, "data/nsw.csv")
```

- O convertir un archivo del disco a otro formato con `convert()`



# PRÁCTICA en clase: las fichas (4): guardar, cargar, importar

Ahora que hemos visto `save()/load()` y `rio`:

- 1 Guardad las fichas en `datos/fichas.RData` (con `save()`); borrad TODO el espacio de trabajo y recuperadlas con `load()`: ¿está todo?
- 2 Exportad las fichas a `datos/fichas.csv` (con `export()` de `rio`), abrid el csv con doble clic en el Explorador (Data Explorer) y volved a importarlo con `import()` a un objeto NUEVO
- 3 Comparad con `str()` el objeto original y el importado: ¿qué le ha pasado al factor? ¿Por qué?



# Valores ausentes (“missing values”): NA

- Muchos conjuntos de datos tienen valores ausentes de ciertas observaciones para algunas variables: ej., descargad (UA Cloud) `earn.RData`
- Sabemos si una observación es NA y la frecuencia total:

```
load("data/earn.RData")  
x <- earn$earnings
```

```
is.na(x)  
table(is.na(x))
```

- Por defecto en R, un cálculo con NAs es NA: debemos decir que los elimine explícitamente (y ser conscientes de lo que implica)

```
mean(x)
```

```
mean(x, na.rm=TRUE)
```

- `na.omit()` elimina observaciones con NAs de una o varias variables

```
earn2 <- na.omit(earn)
```

- ¿Cómo tratar los NAs? Eliminarlos implica selección muestral y la alternativa de imputar valores implica supuestos sobre éstos



# Nota sobre programación “avanzada”

- Como en todo lenguaje de programación R, tiene funciones para
  - ▶ Ejecución condicional `if()`: una parte del código se ejecuta solo si se cumple una condición
  - ▶ Bucles `for()`: se repite un mismo bloque de código mientras se itera por los valores de vector
  - ▶ Crear funciones propias con `function()`
- Una variante de la ejecución condicional, solo para crear variables según una condición

```
data("Affairs", package = "AER")
Affairs$univers <- ifelse(Affairs$education>15, 1, 0)
```